Code

Election Problems and Engine Failure

Foundational Concepts

[I]n this lecture, we're going to start out with a case study, where we look at the preferences of the people and ask the question of "Who is the people's choice?" This example actually is a real election—it was real data from an important election—so it will be interesting for you to see it and make your own decision.

Hide

```
if (!caracas::has_sympy()) {
  caracas::install_sympy()
}
library(caracas)
library(sets)
library(tibble)
library(ggplot2)
library(dplyr)
```

Hide

```
def_sym("X", "Y")

cmStats <- modules::use("CmStats.R")$cmStats
```

This lecture begins as a continuation of the previous lecture by looking at some famous real elections in which we might wonder whether the will of the people prevailed. The challenge of choosing an election winner can be thought of as taking voters' rank orderings of the candidates and returning a societal rank ordering.

Here is a chart that summarizes the voting preferences of the population, giving for each candidate the percentage of the population that ranked the candidate rst, second, and so on.

Hide

```
tibble(
  candidate = c("A", "B", "C", "D", "Total_%"),
  "%_of_1st" = c(40, 13, 18, 29, 100),
  "%_of_2st" = c(14, 46, 18, 22, 100),
  "%_of_1+2" = c(54, 59, 36, 51, 200),
  "%_of_3st" = c(16, 33, 3, 48, 100),
  "%_of_1+2+3" = c(70, 92, 39, 99, 300),
  )
```

candidate <chr>	%_of_1st <dbl>	%_of_2st <dbl>	%_of_1+2 <dbl>	%_of_3st <dbl>	%_of_1+2+3 <dbl>
A	40	14	54	16	70

B	13	46	59	33	92
C	18	18	36	3	39
D	29	22	51	48	99
Total_%	100	100	200	100	300

5 rows

What follows is a summary of how well each candidate would have done under each voting scheme.

• In plurality voting, A would win. • In the vote-for-two scheme, B would win. • Using a run-off, D would win. • Using the Borda Count method, D would win.

What candidate do you think best represents the will of the people? Many people feel that Candidate D is the best choice. Before telling you who actually won this election, let me tell you that this election was a presidential election; thus, there is one more column to be added to the table, namely, the Electoral College votes. In the Electoral College, Candidate A won handily with 180 electoral votes. Interestingly Candidate C, who was not a contender in any of the other schemes presented, actually received the second highest number of electoral votes.

Perhaps now is the time to tell you what election this was: It was the presidential election of 1860. Candidate A is Abraham Lincoln, B is Bell, C is Breckinridge, and D is Douglas. The percentages of people who rated the candidates in second and third places are estimates obtained from historians 72 of the Civil War. It is intriguing to think about the consequences of the statistical issue of how to summarize the data of the voters' opinions.

Hide

```
tibble(
  candidate = c("A", "B", "C", "D", "Winner"),
  plurality = c(.40, .13, .18, .29, "A"),
  Vote_for_two = c(.27, .30, .18, .25, "B"),
  run_off = c(.46, 0., 0., .54, "D"),
  borda_count = c(164, 165, 92, 179, "D"),
  electoral_college = c(180, 39, 72, 12, "A"),
  )
```

candidate <chr>	plurality <chr>	Vote_for_two <chr>	run_off <chr>	borda_count <chr>	electoral_college <chr>
A	0.4	0.27	0.46	164	180
B	0.13	0.3	0	165	39
C	0.18	0.18	0	92	72
D	0.29	0.25	0.54	179	12
Winner	A	B	D	D	A

5 rows

We have seen lots of bad news about election problems. Now, if possible, it gets worse. Next, we will discuss the Condorcet Paradox. Let's look at an example of an election among three candidates, A, B, and C. The following chart summarizes the views of the 30 voters about these candidates.

Hide

```
tibble(rank = c("1_st", "2_st", "3_st"), voters_f_10 = c("A", "B", "C"),
       voters_s_10 = c("B", "C", "A"), voters_t_10 = c("C", "A", "B"))
```

rank <chr>	voters_f_10 <chr>	voters_s_10 <chr>	voters_t_10 <chr>
1_st	A	B	C
2_st	B	C	A
3_st	C	A	B

3 rows

For every candidate, two-thirds of the people have a speci c alternative candidate whom they prefer. We can produce even worse cases—with 10 candidates, for example, when no matter who is declared the winner, there is a specific alternative candidate who is preferred by 90% of the voters.

Many people feel that if there is a speci c candidate who would beat every other candidate in a head-to-head contest, then that candidate, the Condorcet winner, should be declared the victor. Marie Jean Antoine Nicolas Caritat, 73 marquis de Condorcet, wanted to point out a weakness of the Borda Count method. He did so by producing the following famous voters' profile:

Hide

```
tibble(rank = c("1_st", "2_st", "3_st"), "30" = c("A", "B", "C"),
       "10" = c("B", "C", "A"), "10_s" = c("C", "A", "B"), "1" = c("A", "C", "B"),
       "29" = c("B", "A", "C"), "1_s" = c("C", "B", "A")
       )
```

rank <chr>	30 <chr>	10 <chr>	10_s <chr>	1 <chr>	29 <chr>	1_s <chr>
1_st	A	B	C	A	B	C
2_st	B	C	A	C	A	B
3_st	C	A	B	B	C	A

3 rows

B wins with the Borda Count. But A is the Condorcet winner. Thus, the Borda Count method does not always select the Condorcet winner. However, in a sort of voting theory double-reverse, recently, Donald Saari has pointed out an interesting further analysis of this old example. If we rst erased collections of voters whose votes canceled one another, then B should win. Perhaps the Borda Count method chose the right winner.

Hide

```
tibble("20" = c("A", "B", "C"), "0" = c("B", "C", "A"),
       "0_s" = c("C", "A", "B"), "0_t" = c("A", "C", "B"), "28" = c("B", "A", "C"),
       "0_fo" = c("C", "B", "A"))
```

20 <chr>	0 <chr>	0_s <chr>	0_t <chr>	28 <chr>	0_fo <chr>
A	B	C	A	B	C
B	C	A	C	A	B
C	A	B	B	C	A

3 rows

This example shows again the subtleties of summarizing data meaningfully.

All the examples in the last two lectures suggest that summaries of complex situations require contextual arguments to decide among them. Statistical and logical analysis can help a great deal in choosing which arguments to find most persuasive and which systems to use in which settings. Voting theory is an intriguing topic for further study.

Concept and Applications

MULTIPLE LINEAR REGRESSION

When we first started examining the relationship between 2 variables, we used t-tests, which allowed us to determine if there was a statistically significant relationship between 2 variables. We then moved to simple linear regression, which allowed us to fit a line to a predictor versus response. While these are useful in the case where we only have 2 variables, it's more often the case to work with data that has multiple predictors. This lecture is about multiple linear regression.

MULTIPLE LINEAR REGRESSION

Multiple regression is a statistical tool that allows us to examine how multiple predictor variables can be used to model a response variable.

We've generalized the simple linear regression methodology to now describe the relationship between a response variable y and a set of predictorsX1, X2, …, Xn in terms of a linear function.

The variables X1, X2, …, Xp are the p explanatory variables (because they explain what's happening) in the response variable Y.

Y = β0 + β1X1 + β2X2 + … + βpXp + ϵ, where β0, β1, …, βp are the coefficients (parameters), X1, X2,

As in the case of simple linear regression, we want to find the equation of the line that best fits the data. In this case, it means finding (0, 1, …, p) such that the fitted values, ŷi, are as close as possible to the observed values, yi. The following data was collected on a population of women 21 and older of Pima Indian heritage who were living near Phoenix, Arizona. The women were tested for diabetes according to World Health Organization criteria. This is a very rich dataset with the following columns:

- npreg: number of pregnancies;

- glu: plasma glucose concentration in an oral glucose tolerance test;
- bp: diastolic blood pressure (mm Hg);
- skin: triceps skinfold thickness (mm);
- bmi: body mass index (weight in kg/(height in m)2);
- ped: diabetes pedigree function (information on relatives with diabetes);
- age: age in years; and
- type: “Yes” or “No” for diabetic according to WHO criteria.

Hide

```
head(Pima.tr)
```

	npreg	glu	bp	skin	bmi	ped	age	type
	<int>	<int>	<int>	<int>	<dbl>	<dbl>	<int>	<fctr>
1	5	86	68	28	30.2	0.364	24	No
2	7	195	70	33	25.1	0.163	55	Yes
3	5	77	82	41	35.8	0.156	35	No
4	0	165	76	43	47.9	0.259	26	No
5	0	107	60	25	26.4	0.133	23	No
6	5	97	76	27	35.6	0.378	52	Yes

6 rows

Hide

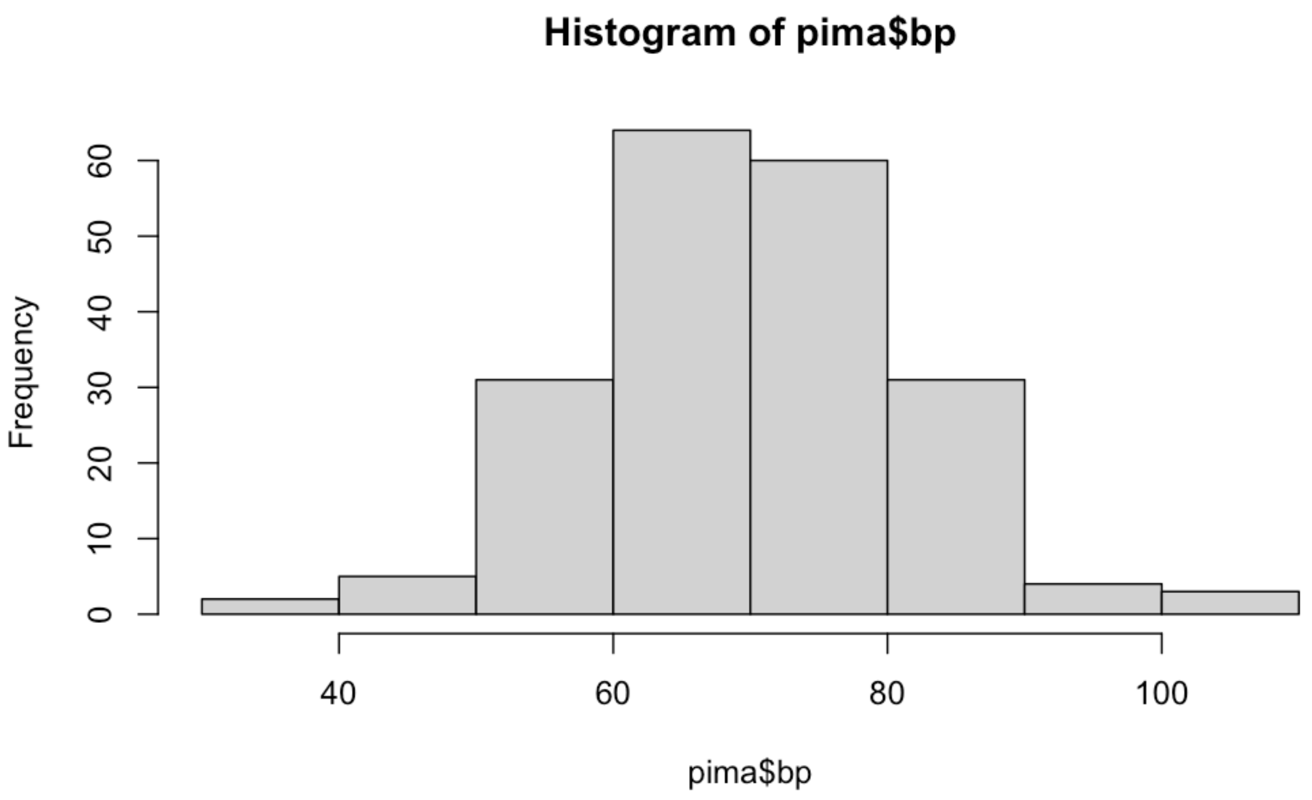
```
pima = Pima.tr
summary(pima)
```

```
    npreg            glu              bp               skin             bmi             
ped        
 Min.   : 0.00   Min.   : 56.0   Min.   : 38.00   Min.   : 7.00   Min.   :18.20   M
in.   :0.0850  
 1st Qu.: 1.00   1st Qu.:100.0   1st Qu.: 64.00   1st Qu.:20.75   1st Qu.:27.57   1
st Qu.:0.2535  
 Median : 2.00   Median :120.5   Median : 70.00   Median :29.00   Median :32.80   M
edian :0.3725  
 Mean   : 3.57   Mean   :124.0   Mean   : 71.26   Mean   :29.21   Mean   :32.31   M
ean   :0.4608  
 3rd Qu.: 6.00   3rd Qu.:144.0   3rd Qu.: 78.00   3rd Qu.:36.00   3rd Qu.:36.50   3
rd Qu.:0.6160  
 Max.   :14.00   Max.   :199.0   Max.   :110.00   Max.   :99.00   Max.   :47.90   M
ax.   :2.2880  
      age         type    
 Min.   :21.00   No :132  
 1st Qu.:23.00   Yes: 68  
 Median :28.00            
 Mean   :32.11            
 3rd Qu.:39.25            
 Max.   :63.00            
```

All of the variables are quantitative except for type, which is categorical (“Yes” for diseased or “No” for non-diseased).

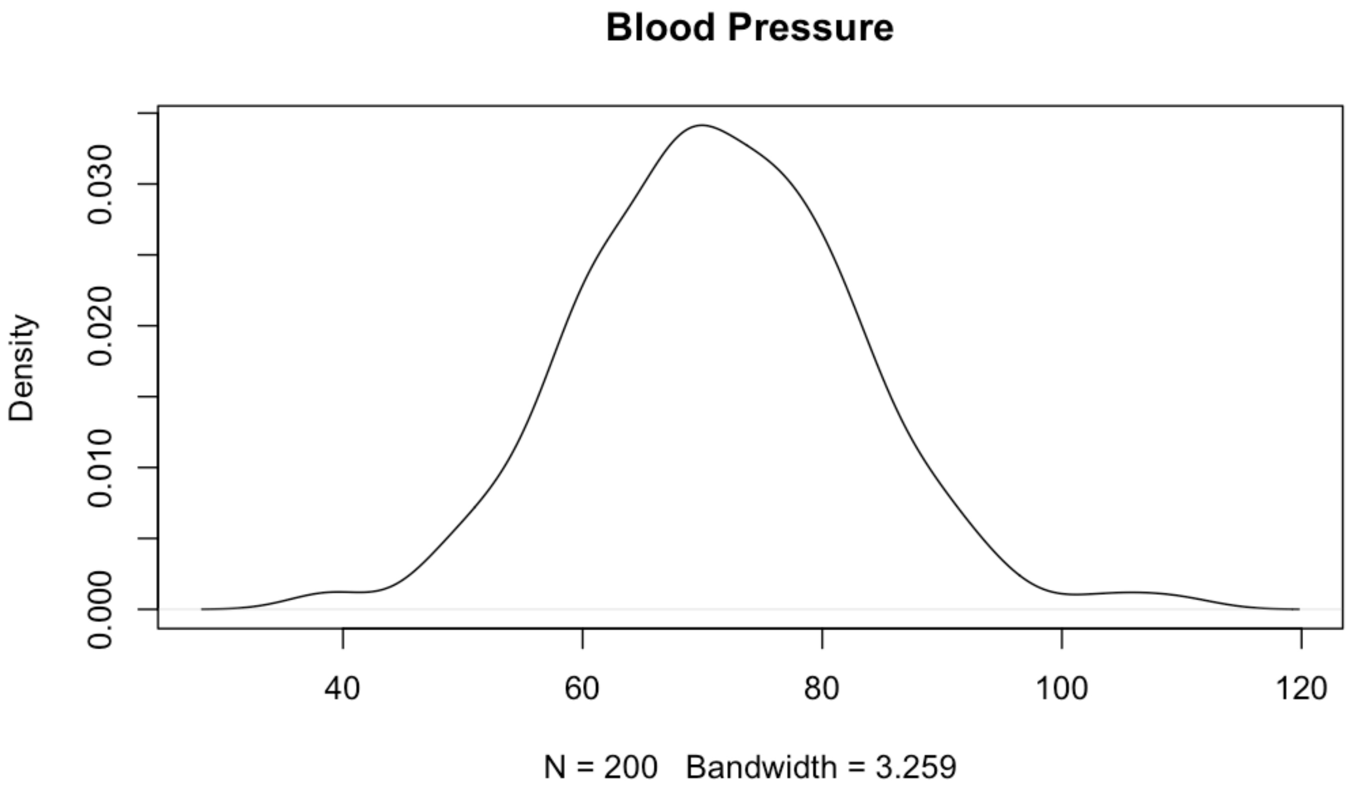
Hide

```
hist(pima$bp)
```



Hide

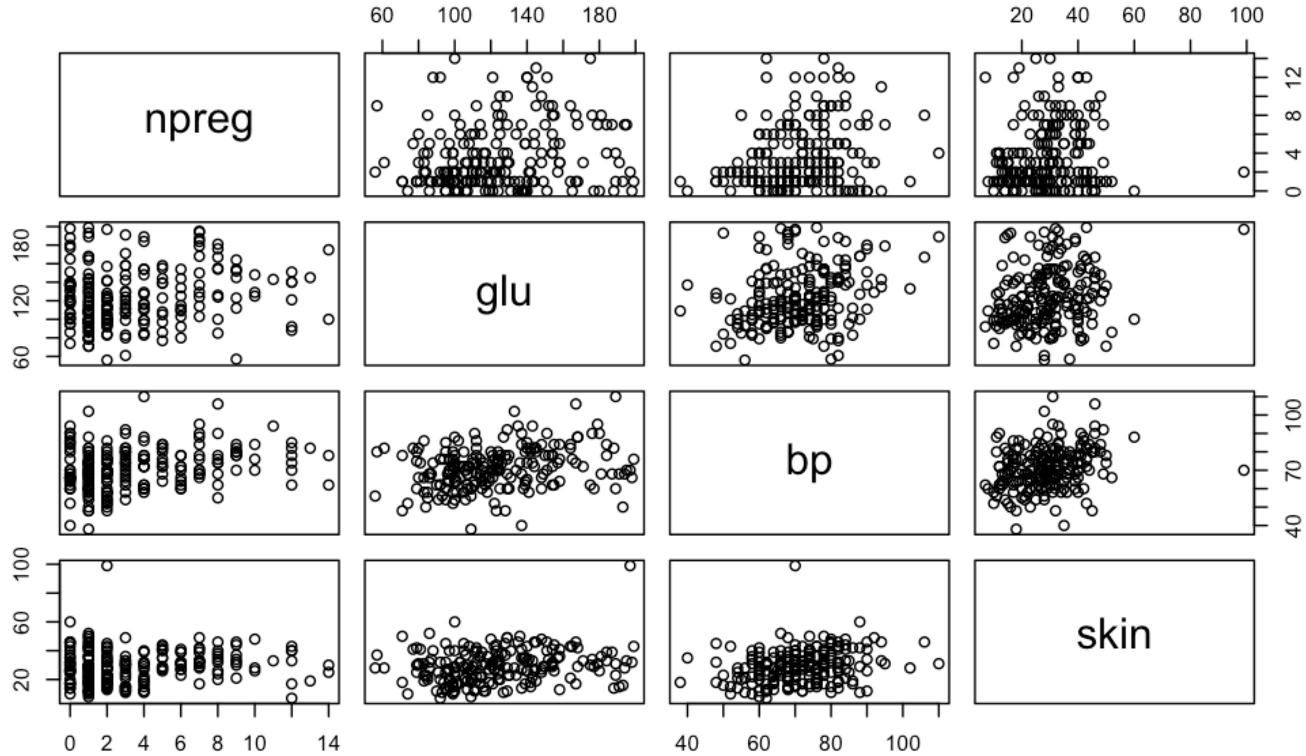
```
plot(density(pima$bp), main= "Blood Pressure")
```



In the following pairs plot of 4 of the data values, there doesn't seem to be a strong relationship in the data values. In fact, skin is a possible outlier.

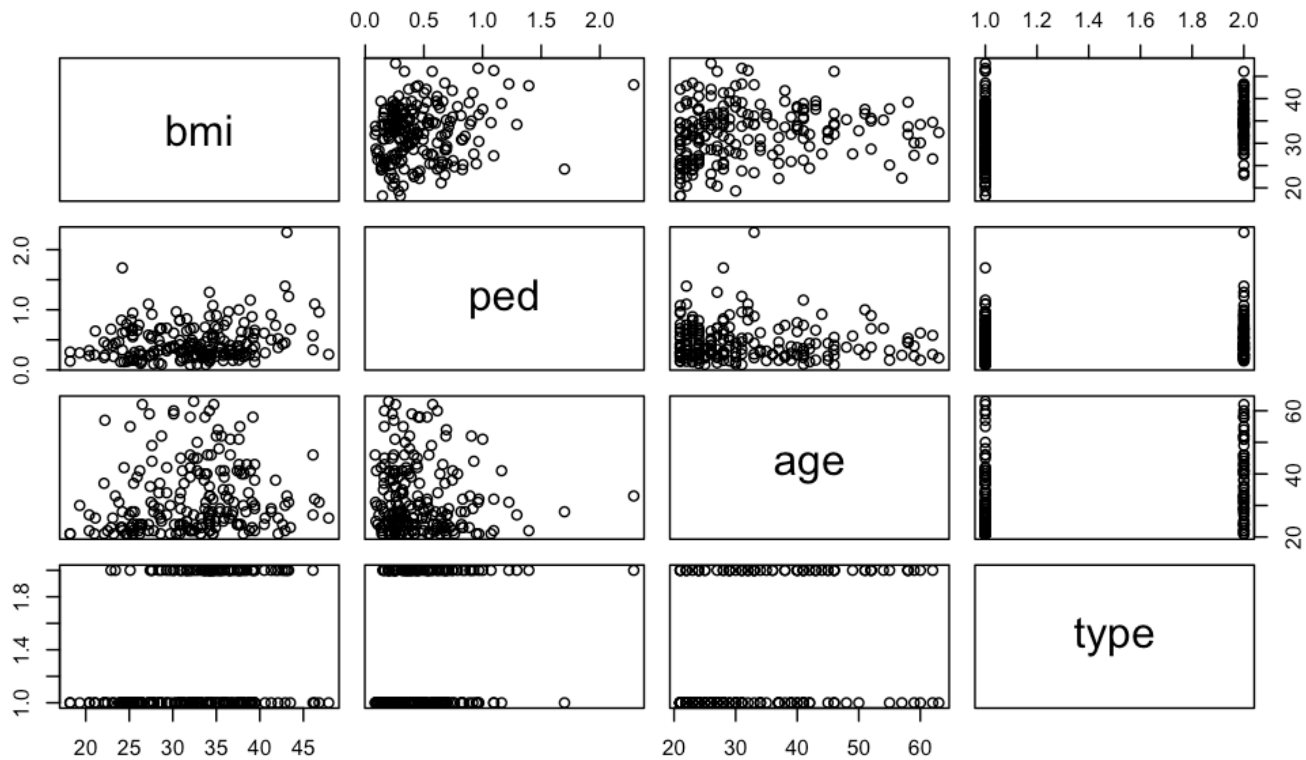
Hide

```
pairs(pima[1:4])
```



Hide

```
pairs(pima[5:8])
```



In the following pairs plot of the other 4 quantitative values, there is a slight relationship between bmi and ped.

Hide

```
round(cor(pima[1:7]), 2)
```

```
      npreg  glu    bp skin  bmi   ped   age
npreg  1.00 0.17  0.25 0.11 0.06 -0.12  0.60
glu    0.17 1.00  0.27 0.22 0.22  0.06  0.34
bp     0.25 0.27  1.00 0.26 0.24 -0.05  0.39
skin   0.11 0.22  0.26 1.00 0.66  0.10  0.25
bmi    0.06 0.22  0.24 0.66 1.00  0.19  0.13
ped   -0.12 0.06 -0.05 0.10 0.19  1.00 -0.07
age    0.60 0.34  0.39 0.25 0.13 -0.07  1.00
```

We can do a correlation analysis of our data to tell us the relationship between our variables. Notice that most of the correlations are at or below 0.34. The highest is between bmi and skin, at 0.66. Then, age and number of pregnancies is correlated at 0.6. That tells us that these variables would go nicely into a linear regression fit. We're not going to see a strong correlation among them.

Let's fit a multiple linear regression model to bmi from the rest of the explanatory variables.

Hide

```
lm1 <-  lm(bmi ~ npreg + glu + bp + skin + ped + age + type, data = pima)
summary(lm1)
```

```

Call:
lm(formula = bmi ~ npreg + glu + bp + skin + ped + age + type, 
    data = pima)

Residuals:
     Min       1Q   Median       3Q      Max 
-19.9065  -2.5723  -0.1412   2.6039  11.2664 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 19.203109   2.346040   8.185 3.69e-14 ***
npreg        0.018970   0.120858   0.157   0.8754    
glu          0.006432   0.011981   0.537   0.5920    
bp           0.046753   0.031272   1.495   0.1366    
skin         0.322989   0.029362  11.000  < 2e-16 ***
ped          2.060288   1.094140   1.883   0.0612 .  
age         -0.061772   0.040459  -1.527   0.1285    
typeYes      1.494968   0.827212   1.807   0.0723 .  
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 4.528 on 192 degrees of freedom
Multiple R-squared:  0.4735,    Adjusted R-squared:  0.4543 
F-statistic: 24.67 on 7 and 192 DF,  p-value: < 2.2e-16
```

We could go through and think hard about which variables to include, or we could automate the process. Stepwise regression lets us drop insignificant variables one by one. This is particularly useful if you have many potential explanatory variables.

Not all of our xi variables are significant predictors. We can drop the most insignificant paremeter out one at a time using the “drop1” command in R.

Hide

```
lm2 <- lm(bmi ~ npreg + glu + bp + skin + ped + age + type, data = pima)
drop1(lm2, test="F")
```

```
Single term deletions

Model:
bmi ~ npreg + glu + bp + skin + ped + age + type
       Df Sum of Sq    RSS    AIC  F value  Pr(>F)
<none>              3937.1 611.98
npreg   1      0.51 3937.6 610.00   0.0246 0.87544
glu     1      5.91 3943.0 610.28   0.2882 0.59201
bp      1     45.83 3982.9 612.29   2.2351 0.13655
skin    1   2481.25 6418.4 707.72 121.0025 < 2e-16 ***
ped     1     72.71 4009.8 613.64   3.5458 0.06121 .
age     1     47.80 3984.9 612.39   2.3311 0.12846
type    1     66.97 4004.1 613.35   3.2661 0.07229 .
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

We can also model glucose as a function of our other predictor variables.

Hide

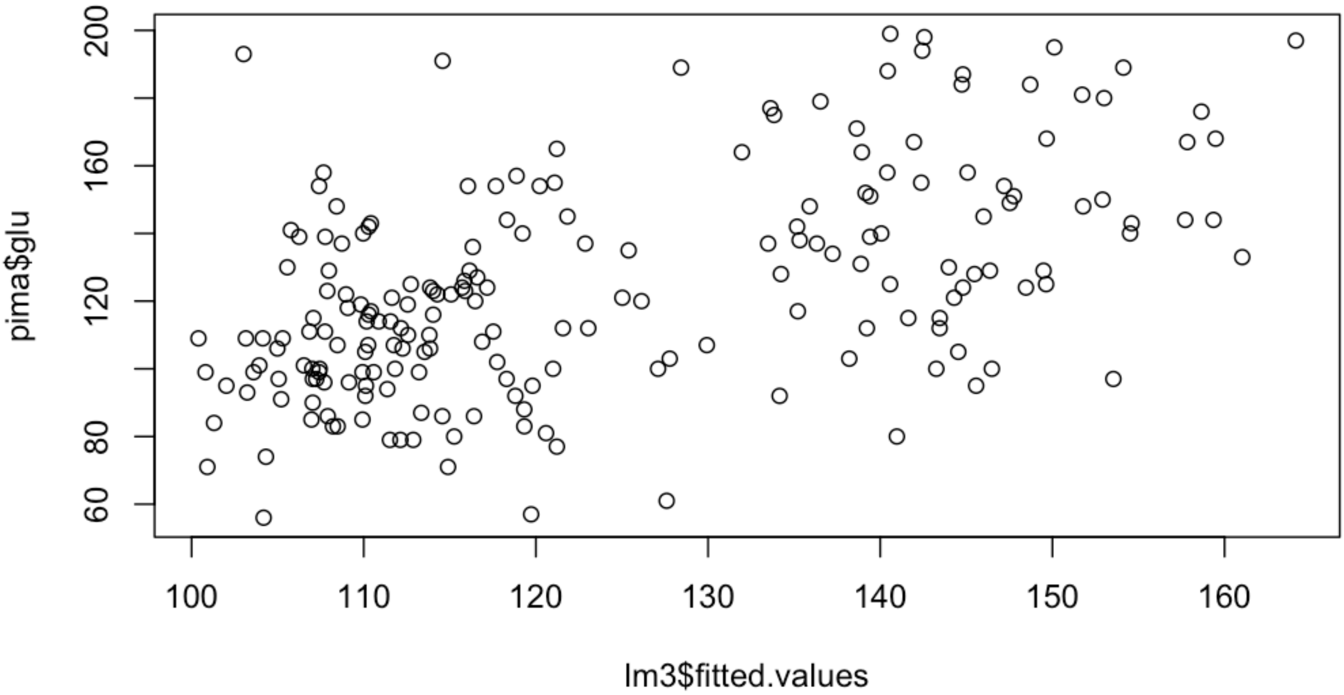
```
lm3 <-  lm(glu ~ bmi + npreg + bp + skin + ped + age + type, data = pima)
summary(lm3)
```

```
Call:
lm(formula = glu ~ bmi + npreg + bp + skin + ped + age + type,
    data = pima)

Residuals:
    Min      1Q  Median      3Q     Max
-66.595 -17.396  -1.641  12.952  89.977

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 68.96159   15.62700   4.413 1.70e-05 ***
bmi          0.23301    0.43405   0.537   0.5920
npreg       -0.84245    0.72494  -1.162   0.2466
bp           0.31786    0.18792   1.691   0.0924 .
skin         0.07046    0.22559   0.312   0.7551
ped         -2.36903    6.64389  -0.357   0.7218
age          0.55832    0.24166   2.310   0.0219 *
typeYes     26.29928    4.64856   5.658 5.51e-08 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 27.26 on 192 degrees of freedom
Multiple R-squared:  0.2853,    Adjusted R-squared:  0.2592
F-statistic: 10.95 on 7 and 192 DF,  p-value: 1.312e-11
```

Plot the actual values against the predicted values. Informally, the model does not appear to be doing a good job.

Hide

```
plot(lm3$fitted.values, pima$glu)
```



Hide

```
predict(lm3)
```

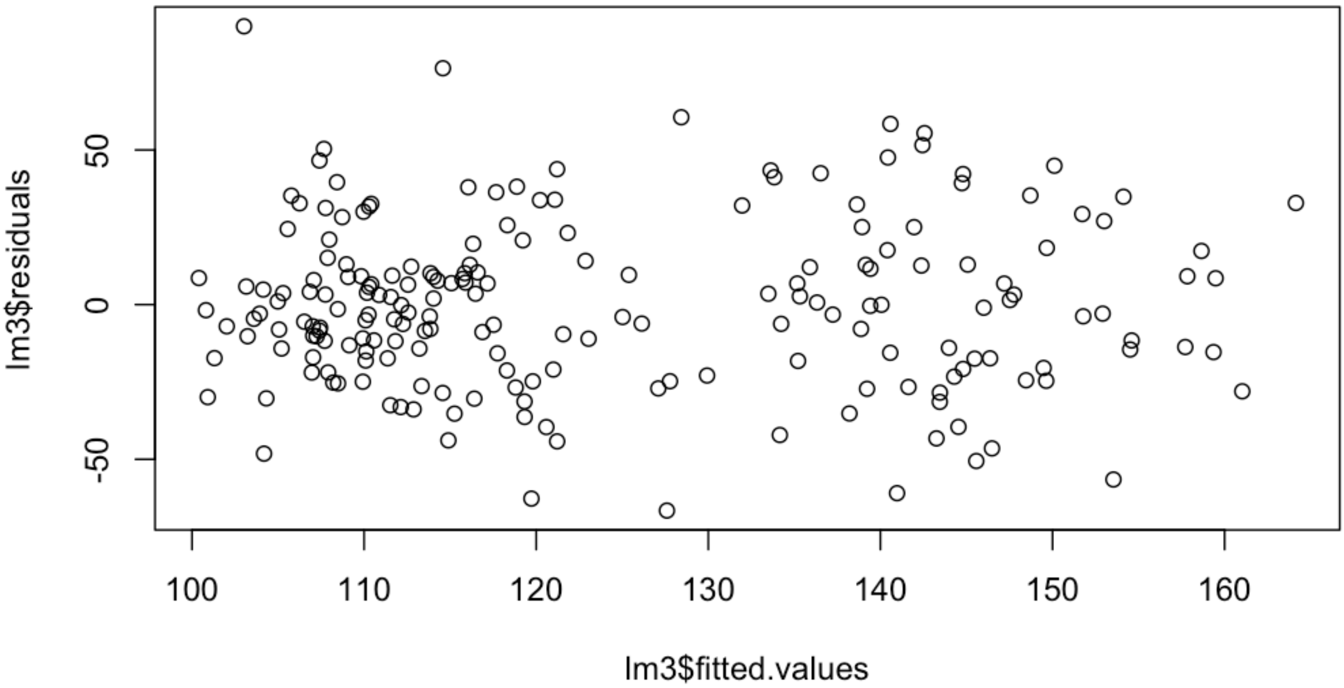
```
       1        2        3        4        5        6        7        8        9
10       11
107.9113 150.1095 121.2165 121.2130 108.4727 153.5411 108.2089 103.0227 135.1703 14
5.4740 133.4885
      12       13       14       15       16       17       18       19       20
21       22
120.2228 154.1226 134.1559 116.4043 107.4087 103.1544 154.5992 147.5264 106.2509 10
9.9259 120.9955
      23       24       25       26       27       28       29       30       31
32       33
119.3260 106.5237 113.3405 138.9335 103.5978 140.0589 116.8786 113.8126 111.5214 10
8.4242 144.2897
      34       35       36       37       38       39       40       41       42
43       44
107.6643 144.5352 121.8302 112.1302 100.9162 117.7512 112.5533 158.6535 107.2404 10
7.9765 105.0584
      45       46       47       48       49       50       51       52       53
54       55
114.5670 112.7330 114.0311 110.0928 138.6324 140.5826 110.2875 108.4849 147.1940 11
1.5525 112.2581
      56       57       58       59       60       61       62       63       64
65       66
116.5941 117.1681 100.4010 107.8831 157.8342 148.7116 109.1409 116.1440 118.8070 10
5.2931 139.4098
      67       68       69       70       71       72       73       74       75
76       77
```

```
137.2332 104.9866 138.8693 125.3857 145.0742 139.2237 151.7286 111.6350 159.4779 15
9.3554 103.9363
      78       79       80       81       82       83       84       85       86
87       88
107.7076 129.9282 125.0206 107.0139 117.6718 149.6342 140.5710 115.0740 110.8691 14
3.4579 110.1696
      89       90       91       92       93       94       95       96       97
98       99
107.0742 105.5610 112.8740 123.0413 152.9125 105.2042 111.8202 154.5120 112.5713 11
1.3697 101.3055
     100      101      102      103      104      105      106      107      108
109      110
151.7832 127.5947 135.2140 113.2124 140.9655 107.4063 127.7725 107.7509 144.8029 11
0.4144 120.5881
     111      112      113      114      115      116      117      118      119
120      121
128.4289 114.0341 138.2012 148.4626 114.8965 136.3195 143.4510 135.9013 116.3331 14
6.0040 103.2325
     122      123      124      125      126      127      128      129      130
131      132
111.7506 139.4186 118.2989 157.7137 121.5701 100.8103 104.1390 126.1328 144.8004 14
9.4842 136.5256
     133      134      135      136      137      138      139      140      141
142      143
115.2543 110.0883 114.5806 119.8008 110.5774 108.7274 107.0438 127.0950 141.9563 15
3.0029 114.2736
     144      145      146      147      148      149      150      151      152
153      154
107.0358 116.4735 116.0529 104.1854 133.6234 115.7183 109.9265 119.3218 139.1459 14
2.5617 140.4359
     155      156      157      158      159      160      161      162      163
164      165
107.7655 149.6645 164.1411 110.2871 115.8472 140.4078 143.9828 107.4420 131.9647 11
0.1271 108.9702
     166      167      168      169      170      171      172      173      174
175      176
106.9638 147.7653 118.3234 117.5237 110.2563 141.6309 113.5333 142.4427 144.7363 14
5.5700 113.8584
     177      178      179      180      181      182      183      184      185
186      187
106.8494 122.8657 119.7222 118.8775 102.0204 119.2236 110.4167 146.4841 115.8898 13
5.3206 143.2565
     188      189      190      191      192      193      194      195      196
197      198
133.8311 104.3186 161.0263 109.8291 121.0842 134.2310 112.1574 109.9684 105.7707 14
6.3697 113.8495
     199      200
109.0741 142.3796
```

We can check for linearity by examining the residual plot.

Hide

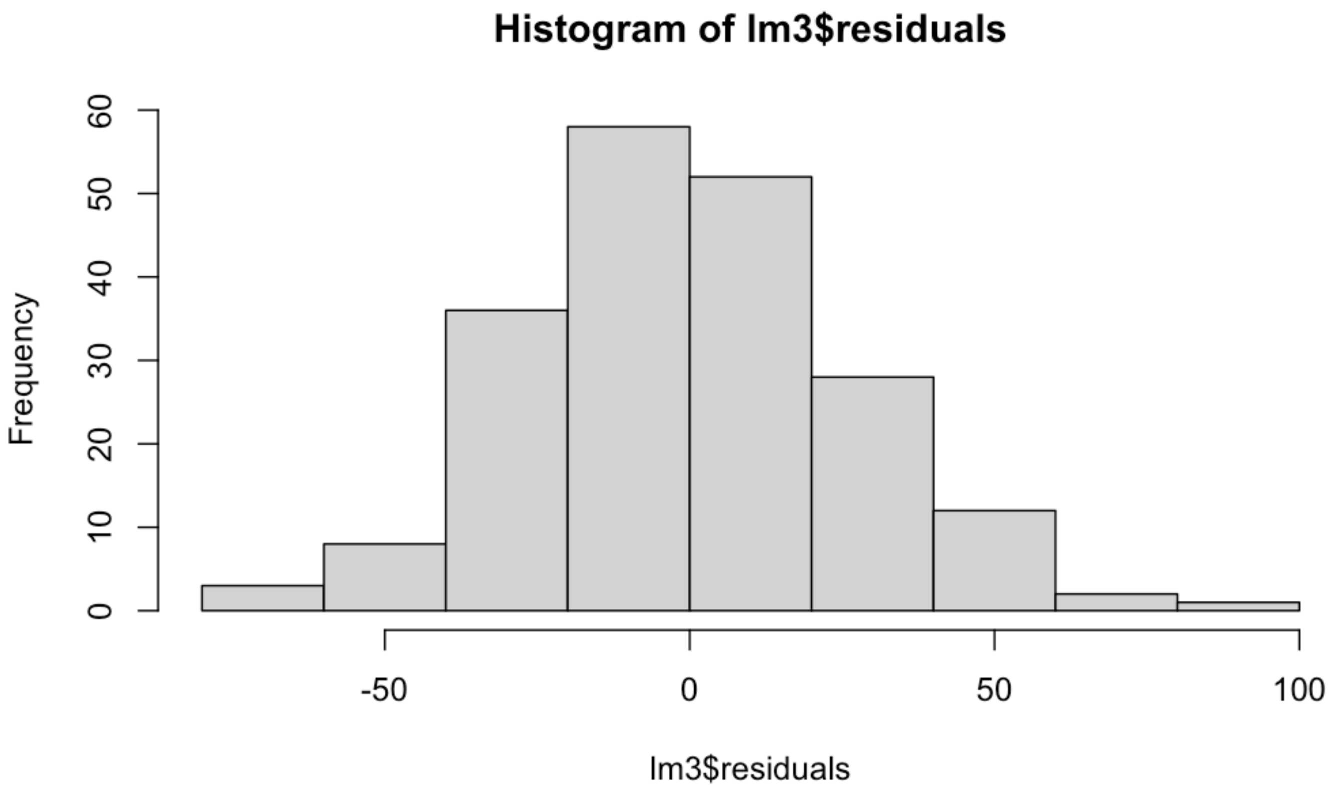
```
plot(lm3$fitted.values, lm3$residuals)
```



Check for normality and constant variance. Do the conditions appear to be approximately satisfied for this model?

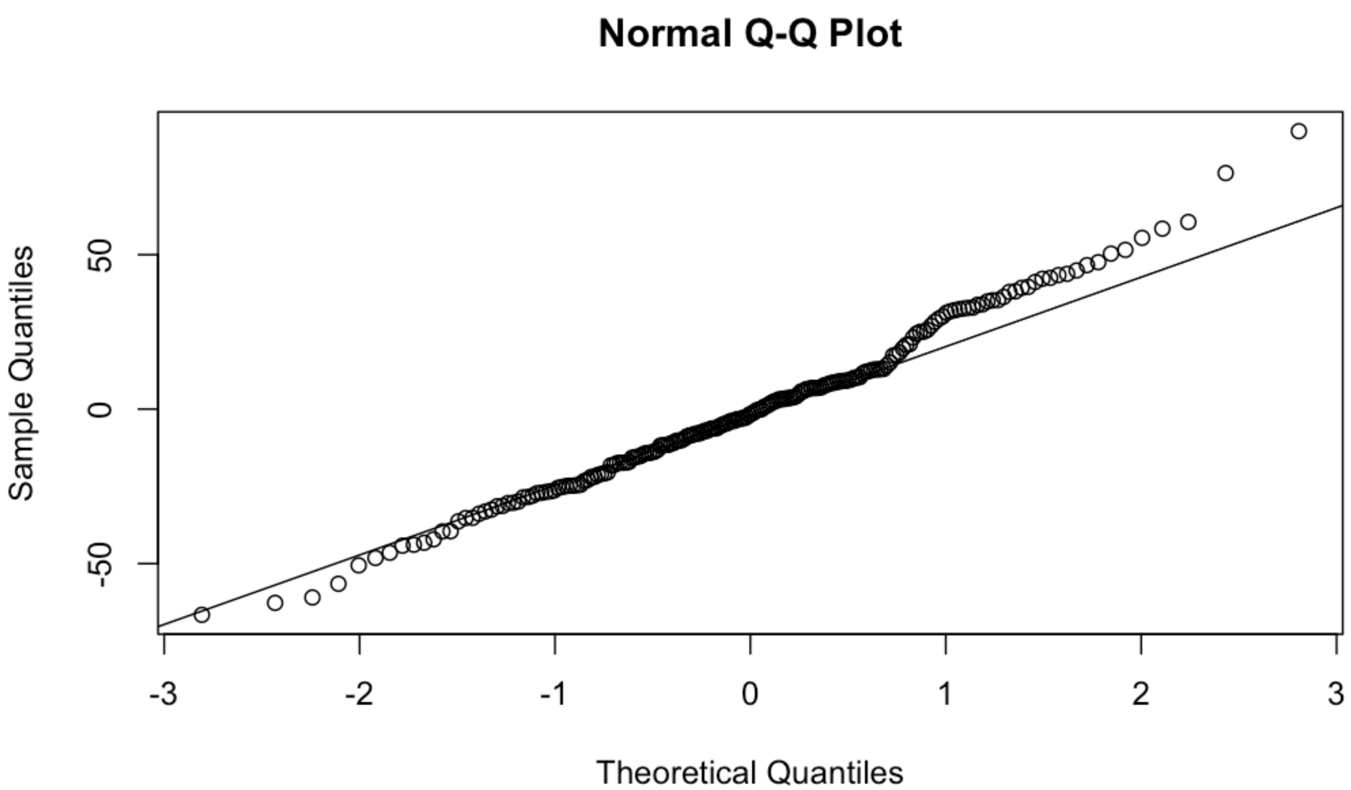
Hide

```
hist(lm3$residuals)
```



Hide

```
qqnorm(lm3$residuals)
qqline(lm3$residuals)
```



DATA WITH HIGHER CORRELATION

The Pima dataset is nice because it has several quantitative predictors and the predictors aren't highly correlated. But what happens when we have predictors that are correlated?

Let's return to the Motor Trend cars data and fit a multiple regression model by including more explanatory variables in a linear regression model.

Let's load our "mtcars" dataset.

Hide

```
mpg_model <-  lm(mpg ~ wt + hp, data=mtcars)
summary(mpg_model)
```

```


Call:
lm(formula = mpg ~ wt + hp, data = mtcars)

Residuals:
   Min     1Q Median     3Q    Max 
-3.941 -1.600 -0.182  1.050  5.854 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept) 37.22727    1.59879  23.285  < 2e-16 ***
wt          -3.87783    0.63273  -6.129 1.12e-06 ***
hp          -0.03177    0.00903  -3.519  0.00145 ** 
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.593 on 29 degrees of freedom
Multiple R-squared:  0.8268,	Adjusted R-squared:  0.8148 
F-statistic: 69.21 on 2 and 29 DF,  p-value: 9.109e-12
```

Let's add the "horsepower" variable to our original model on the mpg dataset. Now our proposed regression model is y = β0 + β1weight + β2horsepower + ϵ, where β0 is our y-intercept, β1 is our regression coefficient for weight, and β2 is our regression coefficient for horsepower.

The coefficient for each of our explanatory variables is the predicted change in y for 1 unit change in x, given that the other explanatory variables are held fixed.

For example, as weight increases by 1 unit, the miles per gallon will decrease by 3.877. Intuitively, this makes sense: The heavier a car gets, the lower miles per gallon it would tend to get.

The p-value for each coefficient tells us whether it's a significant predictor of y given the other explanatory variables in the model. Notice that the p-values for both weight and horsepower are significant.

If explanatory variables are associated with each other, coefficients and p-values will change depending on what else is included in the model.

Hide

```
cor(mtcars$wt, mtcars$hp)
```

```
[1] 0.6587479
```

We see that “hp” and “wt” are correlated with a correlation coefficient of 0.658. It’s possible that these predictors are collinear, meaning that they’re either highly correlated or they contribute the same information to the model.

Because we have several predictor variables in our “mtcars” data, let’s just throw them all in the model. Perhaps this will improve our model even more.

Hide

```
mpg_model = lm(mpg ~., data = mtcars)
summary(mpg_model)
```

```

Call:
lm(formula = mpg ~ ., data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max
-3.4506 -1.6044 -0.1196  1.2193  4.6271

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 12.30337   18.71788   0.657   0.5181
cyl         -0.11144    1.04502  -0.107   0.9161
disp         0.01334    0.01786   0.747   0.4635
hp          -0.02148    0.02177  -0.987   0.3350
drat         0.78711    1.63537   0.481   0.6353
wt          -3.71530    1.89441  -1.961   0.0633 .
qsec         0.82104    0.73084   1.123   0.2739
vs           0.31776    2.10451   0.151   0.8814
am           2.52023    2.05665   1.225   0.2340
gear         0.65541    1.49326   0.439   0.6652
carb        -0.19942    0.82875  -0.241   0.8122
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 2.65 on 21 degrees of freedom
Multiple R-squared:  0.869, Adjusted R-squared:  0.8066
F-statistic: 13.93 on 10 and 21 DF,  p-value: 3.793e-07
```

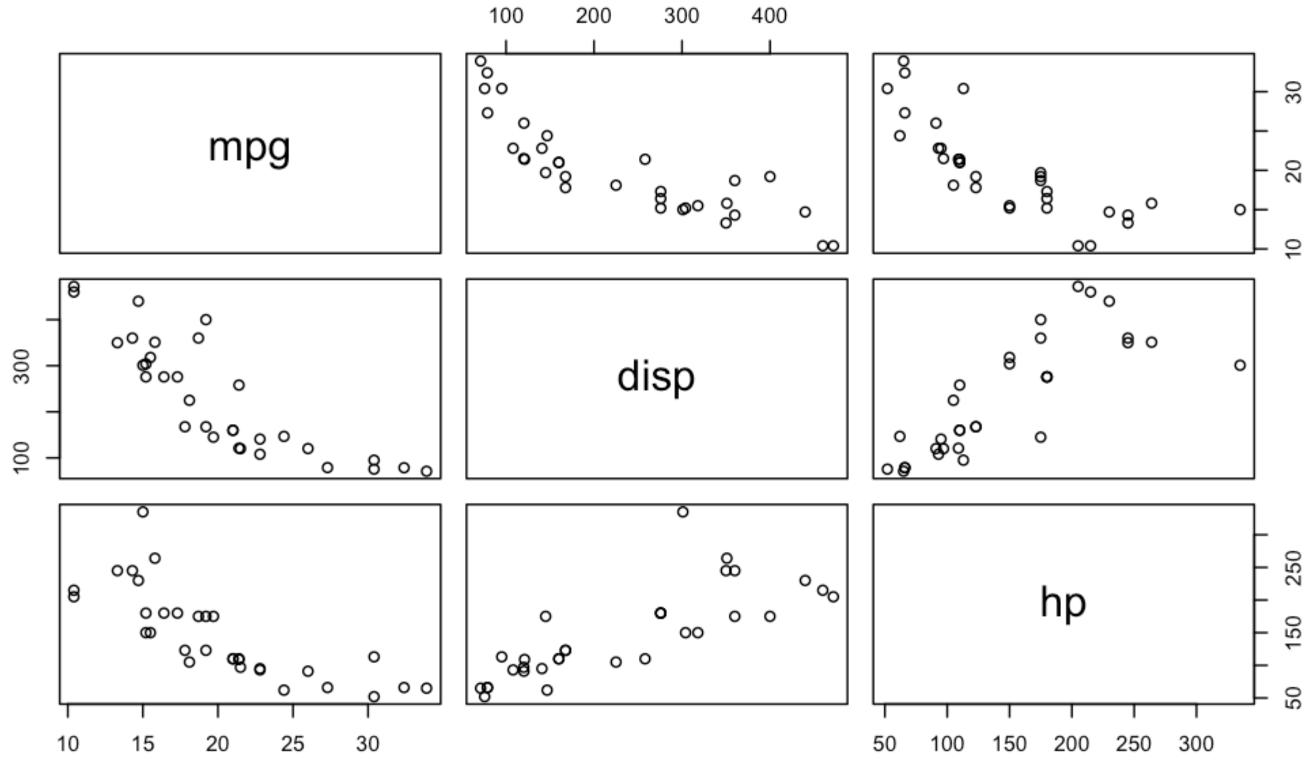
None of our variables are significant. The all have p-values greater than 0.05. In fact, a model using all the variables doesn’t perform as well as our weight-and-horsepower–based model. Our adjusted R2 actually decreased—from 0.8148 to 0.8066.

Adding more explanatory variables will only make R2 increase. The more predictors we have in the model, the more they consume our regression sum of squares. Because R2 is the ratio of the regression sum of squares to the total sum of squares, it will always increase with additional predictor variables.

One problem in our data is that our variables are correlated. We can see this in the pairs plots.

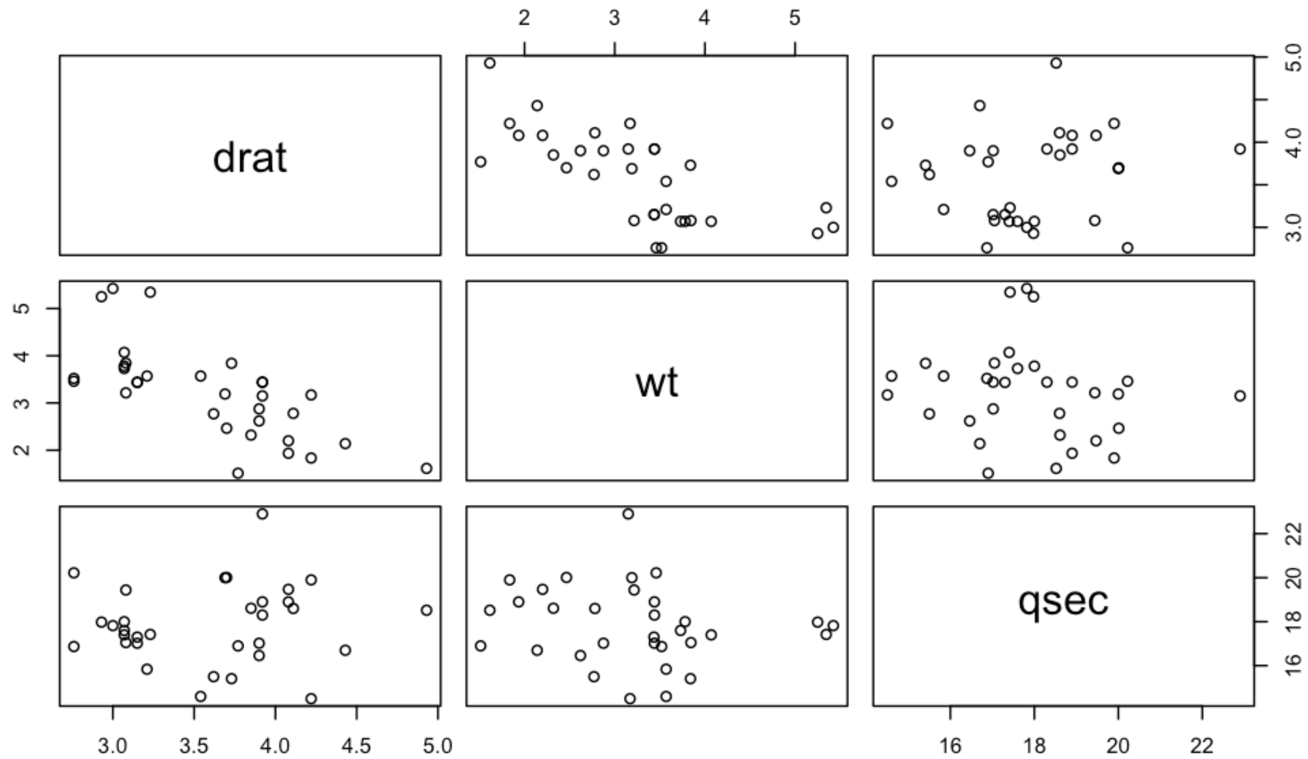
Hide

```
pairs(mtcars[, c(1, 3: 4)])
```



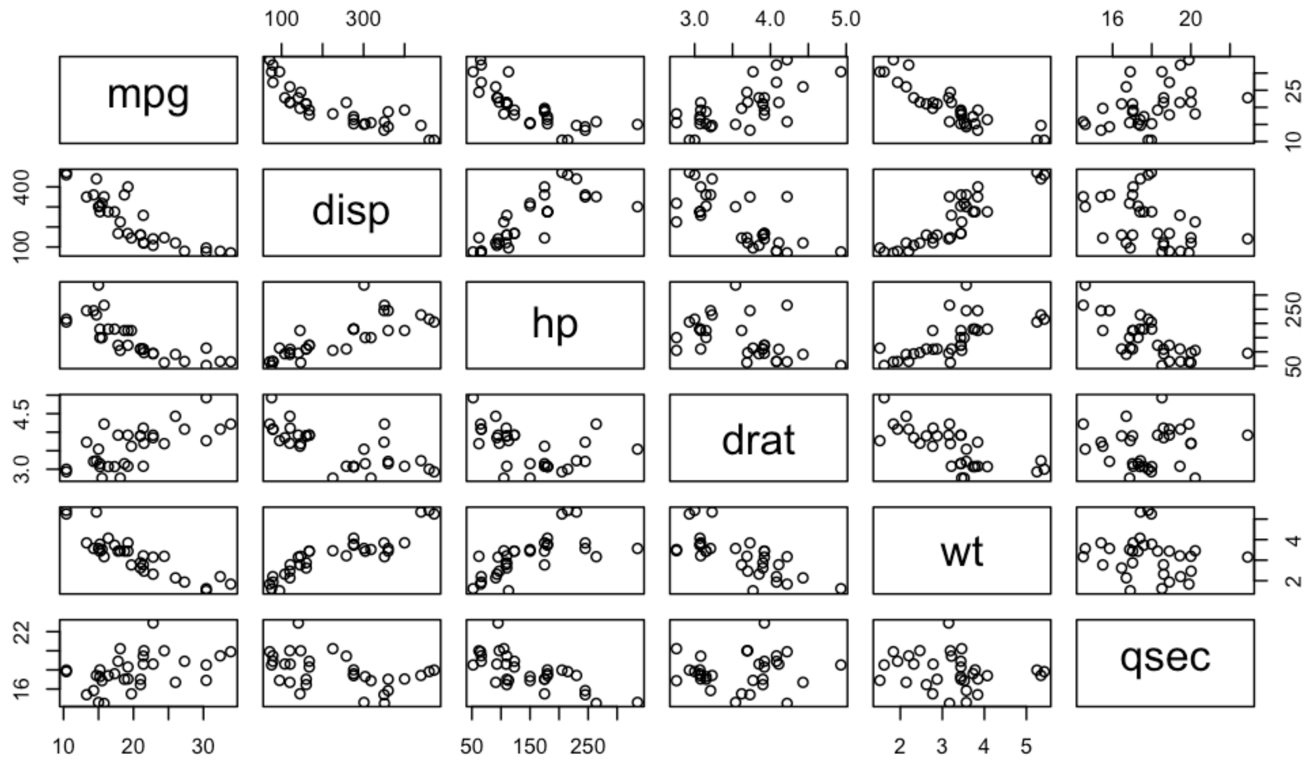
Hide

```
pairs(mtcars[, c( 5: 7)])
```



Hide

```
pairs(mtcars[,c(1,3: 7)])
```



We have highly correlated variables. This leads to model misspecification.

Hide

```
round(cor(mtcars[, c(1:3, 7)]), 2)
```

```
       mpg   cyl  disp  qsec
mpg   1.00 -0.85 -0.85  0.42
cyl  -0.85  1.00  0.90 -0.59
disp -0.85  0.90  1.00 -0.43
qsec  0.42 -0.59 -0.43  1.00
```

In fact, it's possible to overfit a model by including too many explanatory variables. **We have to use the principle of parsimony—the simplest, most efficient model is the best—because the fewer the coefficients have to estimate, the better they will be estimated.**

We can find the best model by pruning. We "step" through the predictor variables and remove the ones that are not significant.

CHOOSING THE BEST MODEL

How do we choose a best model? There are several methods we could use based on which model features are most important to us.

Choose the model with the highest adjusted R2. This assumes that we choose to evaluate the success of our model in terms of the percentage of the variability in the response explained by the explanatory variables.

The p-value for an explanatory variable can be taken as a rough measure for how helpful that explanatory variable is to the model. Insignificant variables may be pruned from the model as long as adjusted R2 doesn't decrease. You can also look at relationships between explanatory variables; if 2 are strongly associated, perhaps both are not necessary.

Let's do a stepwise regression on our linear model fit of miles per gallon with all of our data. This will automatically spit out the best model.

Hide

```
mpg_model2 <-  step(lm(mpg ~., data = mtcars))
```

```
Start:  AIC=70.9
mpg ~ cyl + disp + hp + drat + wt + qsec + vs + am + gear + carb

       Df Sum of Sq    RSS    AIC
- cyl   1    0.0799 147.57 68.915
- vs    1    0.1601 147.66 68.932
- carb  1    0.4067 147.90 68.986
- gear  1    1.3531 148.85 69.190
- drat  1    1.6270 149.12 69.249
- disp  1    3.9167 151.41 69.736
- hp    1    6.8399 154.33 70.348
- qsec  1    8.8641 156.36 70.765
<none>              147.49 70.898
- am    1   10.5467 158.04 71.108
- wt    1   27.0144 174.51 74.280

Step:  AIC=68.92
mpg ~ disp + hp + drat + wt + qsec + vs + am + gear + carb

       Df Sum of Sq    RSS    AIC
- vs    1    0.2685 147.84 66.973
- carb  1    0.5201 148.09 67.028
- gear  1    1.8211 149.40 67.308
- drat  1    1.9826 149.56 67.342
- disp  1    3.9009 151.47 67.750
- hp    1    7.3632 154.94 68.473
<none>              147.57 68.915
- qsec  1   10.0933 157.67 69.032
- am    1   11.8359 159.41 69.384
- wt    1   27.0280 174.60 72.297

Step:  AIC=66.97
mpg ~ disp + hp + drat + wt + qsec + am + gear + carb

       Df Sum of Sq    RSS    AIC
- carb  1    0.6855 148.53 65.121
- gear  1    2.1437 149.99 65.434
```

```
- drat  1    2.2139 150.06 65.449
- disp  1    3.6467 151.49 65.753
- hp    1    7.1060 154.95 66.475
<none>              147.84 66.973
- am    1   11.5694 159.41 67.384
- qsec  1   15.6830 163.53 68.200
- wt    1   27.3799 175.22 70.410

Step:  AIC=65.12
mpg ~ disp + hp + drat + wt + qsec + am + gear

       Df Sum of Sq    RSS    AIC
- gear  1     1.565 150.09 63.457
- drat  1     1.932 150.46 63.535
<none>              148.53 65.121
- disp  1    10.110 158.64 65.229
- am    1    12.323 160.85 65.672
- hp    1    14.826 163.35 66.166
- qsec  1    26.408 174.94 68.358
- wt    1    69.127 217.66 75.350

Step:  AIC=63.46
mpg ~ disp + hp + drat + wt + qsec + am

       Df Sum of Sq    RSS    AIC
- drat  1     3.345 153.44 62.162
- disp  1     8.545 158.64 63.229
<none>              150.09 63.457
- hp    1    13.285 163.38 64.171
- am    1    20.036 170.13 65.466
- qsec  1    25.574 175.67 66.491
- wt    1    67.572 217.66 73.351

Step:  AIC=62.16
mpg ~ disp + hp + wt + qsec + am

       Df Sum of Sq    RSS    AIC
- disp  1     6.629 160.07 61.515
<none>              153.44 62.162
- hp    1    12.572 166.01 62.682
- qsec  1    26.470 179.91 65.255
- am    1    32.198 185.63 66.258
- wt    1    69.043 222.48 72.051

Step:  AIC=61.52
mpg ~ hp + wt + qsec + am

       Df Sum of Sq    RSS    AIC
- hp    1     9.219 169.29 61.307
<none>              160.07 61.515
- qsec  1    20.225 180.29 63.323
- am    1    25.993 186.06 64.331
- wt    1    78.494 238.56 72.284
```

```
Step:  AIC=61.31
mpg ~ wt + qsec + am

       Df Sum of Sq    RSS    AIC
<none>              169.29 61.307
- am    1    26.178 195.46 63.908
- qsec  1   109.034 278.32 75.217
- wt    1   183.347 352.63 82.790
```

The output from our stepwise regression model is as follows. This is the best model.

Hide

```
summary(mpg_model2)
```

```


Call:
lm(formula = mpg ~ wt + qsec + am, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4811 -1.5555 -0.7257  1.4110  4.6610 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)   9.6178     6.9596   1.382 0.177915    
wt           -3.9165     0.7112  -5.507 6.95e-06 ***
qsec          1.2259     0.2887   4.247 0.000216 ***
am            2.9358     1.4109   2.081 0.046716 *  
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 2.459 on 28 degrees of freedom
Multiple R-squared:  0.8497,	Adjusted R-squared:  0.8336 
F-statistic: 52.75 on 3 and 28 DF,  p-value: 1.21e-11
```